AI-Based Motor Fault Detection Using Analog Current Sensing

[Author 1 Name]
[Department / School]
[Institute Name]
[City, Country]
[email address]

[Author 2 Name]
[Department / School]
[Institute Name]
[City, Country]
[email address]

[Author 3 Name]
[Department / School]
[Institute Name]
[City, Country]
[email address]

[Guide Name]*
[Department / School]
[Institute Name]
[City, Country]
[email address]

***Abstract*—Unplanned motor failures cause downtime and costly repairs, and most condition-monitoring systems depend on vibration or temperature sensors that add cost and mounting effort. This paper presents a low-cost system that detects motor faults from a single signal, the current drawn by the motor, using analog signal conditioning followed by machine learning. A 0.1 Ω shunt resistor converts the motor current into a voltage, an LM358 differential amplifier (gain 20) raises it to the volt range, and a unity-gain Sallen-Key low-pass filter (232 Hz) removes commutator ripple before a 10-bit Arduino Uno ADC samples it at 1 kHz. A Python program cuts the stream into one-second windows, extracts eleven time- and frequency-domain features, and classifies the motor as normal, overloaded or mechanically faulty with a Random Forest of 100 trees. The complete chain was evaluated in a software-in-the-loop simulation that reproduces the circuit transfer functions, the ADC quantization and the motor behavior, and is streamed to a live web dashboard. On 300 held-out windows the classifier reached 100% accuracy, faults were detected after 0.87–1.29 s on average, and no false alarm occurred in 12 min of healthy operation. Removing the analog filter increased the current-measurement error from 3.2 mA to 16.1 mA and reduced accuracy to 92.3%. Because the data are simulated, validation on hardware is the next step.**

***Keywords*— Motor fault detection, current signature analysis, analog signal conditioning, Sallen-Key filter, Random Forest, Arduino**

I. INTRODUCTION

Electric motors drive pumps, fans, conveyors and machine tools, and an unexpected motor failure stops the whole process that depends on it. Conventional maintenance either replaces parts on a fixed schedule or waits for a breakdown, whereas condition-based maintenance watches the machine and acts when its health deteriorates [1]. Condition monitoring of motors has traditionally relied on vibration, temperature and acoustic sensors, which must be mounted on the machine, need calibration and increase the cost [2-3]. Motor current signature analysis (MCSA) is an attractive alternative because mechanical and electrical faults modulate the current that the motor already draws, so one current measurement carries information about the whole drive [4-5].

Recent studies combine current signals with signal processing and machine learning to classify motor faults automatically [6-7]. Most of this work targets induction motors and assumes that a clean, well digitized signal is already available from a data-acquisition card. In low-cost systems this assumption does not hold. A shunt resistor produces only millivolts, an amplifier can saturate, brush noise and commutator ripple contaminate the signal, and a microcontroller ADC with limited resolution aliases anything that is not filtered before sampling. The analog front-end therefore decides how much of the fault information reaches the classifier.

These observations motivate a system that treats analog signal conditioning as a first-class part of the design. This paper presents an AI-based fault detector for a small 12 V geared DC motor that uses one signal, the motor current. A 0.1 Ω shunt resistor, an LM358 differential amplifier and a Sallen-Key low-pass filter condition the signal before an Arduino Uno digitizes it. A Python program extracts eleven features from one-second windows and a Random Forest classifies the motor as normal, overloaded or mechanically faulty, while a live web dashboard shows the verdict and the signals behind it. Because the hardware has not been built yet, the whole chain is evaluated in a software-in-the-loop simulation that reproduces the circuit transfer functions and the ADC quantization exactly.

The primary research objectives of this study are as follows:

1. *Analog Signal Conditioning:* To design a shunt, differential-amplifier and Sallen-Key filter chain

that converts the motor current into a clean voltage matched to the 0–5 V range of a 10-bit ADC.

2. *Single-Signal Diagnosis:* To classify the normal, overload and mechanical-friction conditions from the current alone, using eleven features and a Random Forest.

3. *Real-Time Operation:* To keep feature extraction and inference well inside the 50 ms update interval on a laptop CPU and to stream the result to a web dashboard.

4. *Pre-Hardware Validation:* To measure accuracy, detection latency, false alarms and robustness in simulation before building the hardware, and to quantify what each analog stage contributes.

The main contributions are an analog front-end whose gain, bandwidth and headroom are derived from the motor's current range, a compact feature set and classifier evaluated for accuracy, latency, false alarms and robustness, and a measurement of what the analog filter is worth, obtained by removing it. The rest of the paper is organized as follows: Section II reviews related work, Section III describes the design and implementation, Section IV presents the results and Section V concludes the paper.

II. RELATED WORK

Motor fault diagnosis has been studied for several decades. This section summarizes the approaches that are most relevant to a current-based, low-cost system and points out the gaps that the proposed design addresses.

A. Current Signature Analysis in Motors

MCSA analyzes the stator or armature current of a running machine to find fault signatures without any sensor on the motor itself. Early work showed that faults such as broken rotor bars and eccentricity produce characteristic components in the current spectrum of induction motors [4-5]. Later reviews extended the approach to electrical machines, power electronics and drives and highlighted the role of advanced signal processing [2-3]. In a brushed DC motor the physics differs but the principle is the same: the armature current is proportional to the shaft torque, so a change in mechanical load changes the mean current, while a local rub adds a disturbance that repeats once per revolution. The commutator also produces a ripple at the number of segments times the shaft speed, which a measurement system must suppress rather than read as a fault.

B. Machine Learning for Fault Diagnosis

Machine learning replaces hand-set thresholds with a classifier learned from labeled data. Reviews describe two families: classical models, such as support vector machines, decision trees and ensembles, that work on engineered features, and deep networks that learn features directly from raw signals [6-7]. Deep learning gives excellent results on large bearing data sets but needs large amounts of data and computing power and is hard to explain. Random Forests [8] average many decision trees trained on random subsets of the data and features. They work with small data sets, need no feature scaling, run quickly and provide feature-importance scores that make the decision easier to explain, which suits a laboratory-scale system. The scikit-learn library [9] provides a tested implementation.

C. Analog Front-Ends and Data Acquisition

Whatever the classifier, the measurement chain must deliver a faithful signal. A low-side shunt is the simplest current sensor, and the LM358 is a common single-supply amplifier whose input range includes ground, which allows low-side sensing without a negative supply [10-11]. A differential amplifier rejects the noise common to both inputs, and a Sallen-Key stage gives a second-order low-pass response with one op-amp [12-13]. Sampling theory requires that components above half the sampling rate be removed before conversion, otherwise they alias into the band of interest [14], and spectra of short records depend on the window function [15]. Many learning studies treat the acquisition chain as given; this work makes it explicit and measures its effect. Few low-cost designs connect strong signal-processing and learning methods with a documented analog front-end that a student can build on a breadboard; the proposed detector, about ₹1,740–2,840 in parts, fills this gap with every stage specified by component values and evaluated before the hardware is built.

III. DESIGN AND IMPLEMENTATION

A. System Architecture

The system follows the signal from the motor to the operator, as shown in Fig. 1 and Fig. 2. The 12 V motor drives a pulley and belt rig that can apply a load or a friction brake. The motor returns its current to ground through a 0.1 Ω shunt resistor. The small voltage across the shunt is amplified by the first half of an LM358, filtered by the second half, and sampled by the 10-bit ADC of an Arduino Uno at 1 kHz on pin A0 [16]. The Arduino streams the counts over USB. On the laptop, a Python program converts the counts back to amperes, keeps a one-second sliding window that advances every 50 ms, extracts eleven features and lets a Random Forest of 100 trees vote on the condition. The smoothed result is sent through a WebSocket [17] to a web dashboard that displays the waveform, the spectrum, the fault status and the confidence.

Fig. 2 shows the run-time loop that repeats every 50 ms. In this work the motor, the circuit and the Arduino are replaced by a simulator that produces the same ADC counts, so everything to the right of the ADC in Fig. 1 is the real software that will run with the hardware.

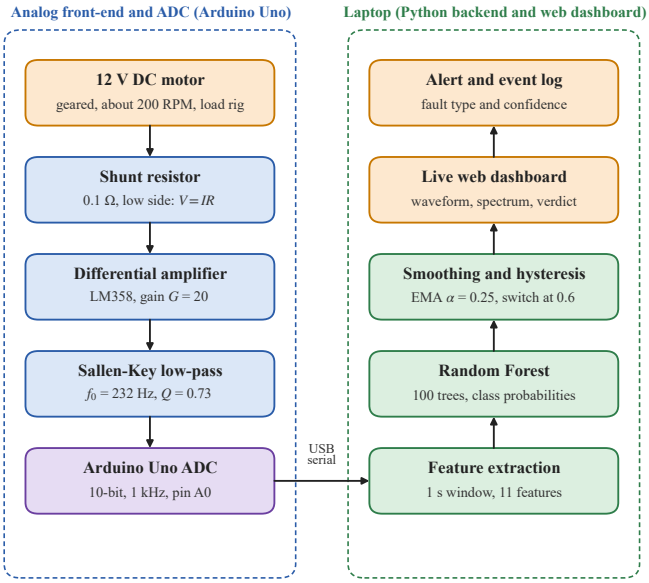


Fig. 1. Block diagram of the proposed system model

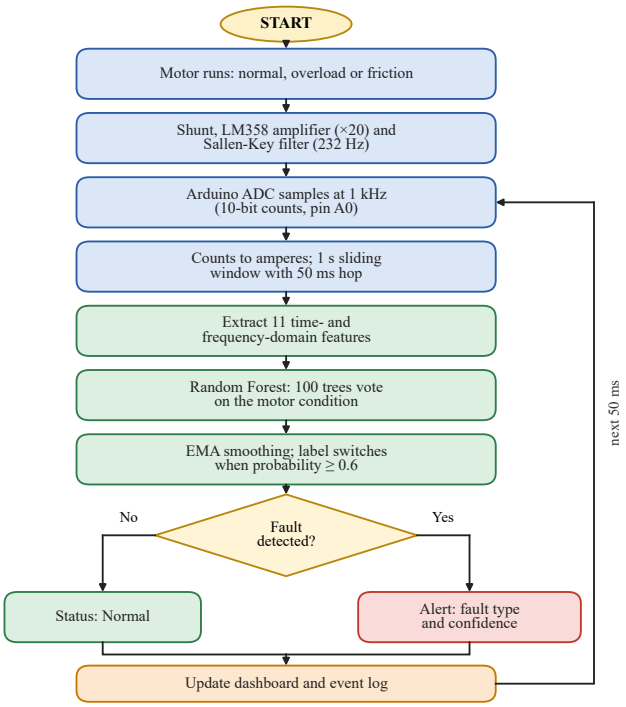


Fig. 2. Flowchart of the proposed system methodology

B. *Hardware Implementation*

The components are listed in Table I. Each contributes to sensing, conditioning, digitizing or displaying the motor current, and the complete analog circuit is shown in Fig. 3.

1. Shunt Resistor

The shunt converts the motor current into a voltage, as given in Equation (1). It is placed on the low side so that one terminal is at ground, which suits the single-supply LM358.

TABLE I. HARDWARE COMPONENTS AND THEIR USAGE

S.No	Components	Usage
1.	Arduino Uno R3	Samples the filtered voltage on A0 (10-bit, 1 kHz), sends the counts over USB and supplies 5 V to the op-amp.
2.	12 V geared DC motor	The machine under test (about 200 RPM at the output shaft). Its current is the only signal measured.
3.	Shunt resistor	0.1 Ω, 2 W. Converts the motor current into a small voltage on the low side.
4.	LM358 op-amp	One half is the differential amplifier (gain 20), the other half the Sallen-Key low-pass filter.
5.	Resistors and capacitors	10 kΩ, 200 kΩ, 100 nF and 47 nF set the gain and the 232 Hz cutoff.
6.	12 V, 2 A adapter	Powers the motor. Its ground is joined to the Arduino ground.
7.	Pulley, belt and brake	Applies a controlled load or friction to the motor shaft.
8.	Laptop (Python)	Extracts features, runs the Random Forest and hosts the web dashboard.

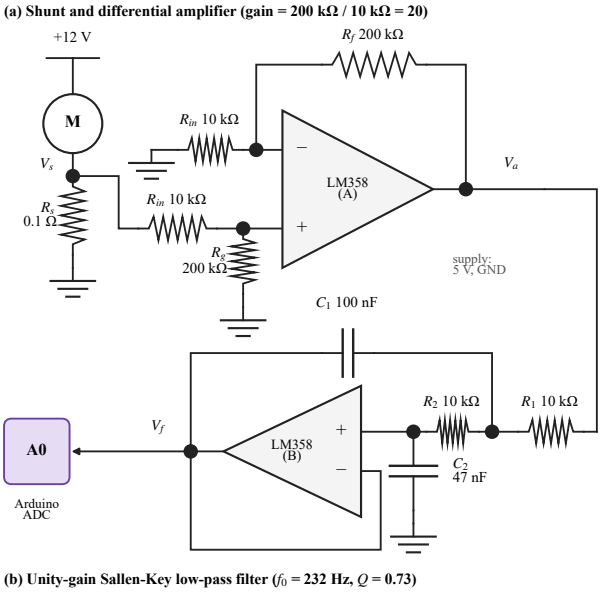


Fig. 3. Circuit schematic of the analog front-end

Shunt voltage:

$$V_s = IR_s \qquad (1)$$

Where:

- V_s = Voltage across the shunt (V)
- I = Motor current (A)
- R_s = Shunt resistance (0.1 Ω)

At the free-running current of 0.33 A the shunt voltage is 33 mV and the dissipation is only 11 mW. Even at 1 A the shunt dissipates 0.1 W, far below its 2 W rating, and the 33 mV drop is 0.28% of the 12 V supply.

2. Differential Amplifier

The first half of the LM358 is a differential amplifier built from two 10 kΩ input resistors, a 200 kΩ feedback resistor and a 200 kΩ resistor from the non-inverting input to ground. Its gain is set by the resistor ratio (Equation (2)).

$$V_a = GV_s, \quad G = \frac{R_f}{R_{in}} = 20 \tag{2}$$

The LM358 runs from the 5 V rail of the Arduino, where its output swings only up to about 3.5 V (V_{sat}). The largest measurable current therefore follows from Equation (3).

$$I_{\max} = \frac{V_{\text{sat}}}{GR_s} = 1.75\ \text{A} \tag{3}$$

This limit is well above the 0.33 A free-running current and the 0.95 A mean current at full simulated overload, so the amplifier stays linear in every condition.

3. Sallen-Key Low-Pass Filter

The second half of the LM358 forms a unity-gain Sallen-Key low-pass filter with $R_1 = R_2 = R = 10$ kΩ, $C_1 = 100$ nF in the feedback path and $C_2 = 47$ nF to ground. Its natural frequency and quality factor follow from Equations (4) and (5), and its transfer function is given by Equation (6).

Natural frequency and quality factor:

$$f_0 = \frac{1}{2\pi R\sqrt{C_1 C_2}} \tag{4}$$

$$Q = \frac{1}{2}\sqrt{\frac{C_1}{C_2}} \tag{5}$$

Transfer function:

$$H(s) = \frac{1}{s^2R^2C_1C_2 + 2sRC_2 + 1} \tag{6}$$

With these values $f_0 = 232$ Hz and $Q = 0.73$, and the −3 dB point lies at 239 Hz. Fig. 4 shows the response. The fault signatures lie below 50 Hz and pass with less than 0.02 dB of loss, while the 600 Hz commutator ripple, the product of the 30:1 gear ratio, six commutator segments and the 3.3 rev/s shaft speed, is attenuated by 16.5 dB.

A second-order filter at 232 Hz gives only 13.4 dB of attenuation at the 500 Hz Nyquist frequency, so it is a compromise: the cutoff must stay well above the fault signatures, and the 5% ripple is reduced to about one to three ADC counts depending on the load.

4. ADC and Sampling

The filtered voltage V_f is digitized by the 10-bit ADC of the Arduino Uno with a 5 V reference at 1 kHz

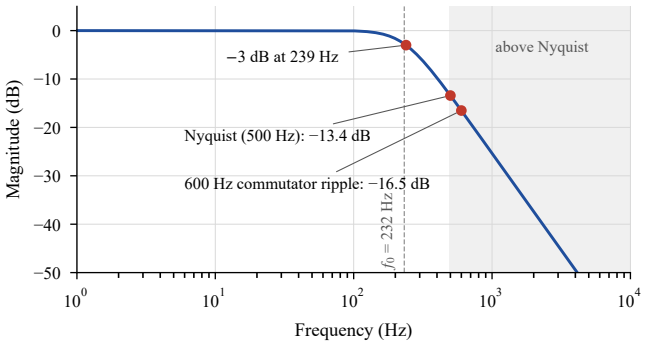


Fig. 4. Frequency response of the Sallen-Key low-pass filter

(Equation (7)). One count equals 4.89 mV at the ADC input, which is 2.44 mA of motor current, and the laptop recovers the current with Equation (8), where N is the count and $V_{\text{ref}} = 5$ V.

$$N = \text{round}\left(\frac{1023\,V_f}{V_{\text{ref}}}\right) \tag{7}$$

$$I = \frac{N\,V_{\text{ref}}}{1023\,GR_s} \tag{8}$$

The free-running motor therefore produces about 136 counts and the overloaded motor about 328 counts.

C. Software Implementation

The software has three layers: a simulator that stands in for the hardware, a feature extractor with a classifier, and a real-time web dashboard.

1. Motor and Circuit Simulation

The simulator reproduces the chain of Fig. 3 stage by stage. A motor model sets the armature current to $I_0 + k\tau$, where $I_0 = 0.30$ A is the no-load current, τ is a normalized load torque and $k = 0.65$ A per unit of torque. The shaft speed falls by up to 45% as the load rises from about 200 RPM, and the motor settles to a new operating point with a 0.4 s time constant. The model adds the effects that a real motor shows: a 4% current ripple once per revolution from pulley eccentricity, slow random drift of the load, a 5% commutator ripple at 600 Hz and short brush-arcing spikes.

The three conditions come from two hidden parameters, listed in Table II: the load torque and the intensity of a rub. In the friction condition a current spike of up to 0.35 A times the rub intensity occurs once per revolution, together with 5–40 Hz stick-slip chatter. The analog stages run at ten times the ADC rate (10 kHz). The shunt, the ×20 gain with 20 mV rms of noise and the clipping at 3.5 V are modeled in the time domain, and the Sallen-Key filter is implemented from the exact transfer function of Equation (6) with the bilinear transform, carrying its internal state from one block to the next like the charge on real capacitors. The ADC keeps every tenth sample and quantizes it to 10 bits. Only these counts reach the classifier, exactly as an Arduino would send them.

TABLE II. SIMULATED MOTOR CONDITIONS

Condition	Load torque τ	Rub intensity	Visible effect
Normal	0.05	0	free running
Overload	0.25 + 0.75 s	0	high mean current, slower shaft
Friction	0.10 + 0.25 s	0.3 + 0.7 s	spike once per revolution

2. Feature Extraction

Every 50 ms the latest 1000 samples are converted to amperes and summarized by the eleven features of Table III. The time-domain features describe level and spikiness: the RMS value (Equation (9), where x_k are the L = 1000 samples), the crest factor (peak divided by RMS) and the kurtosis. The frequency-domain features come from an FFT of the Hann-windowed signal, zero-padded to four times its length: the frequency and amplitude of the largest peak between 0.75 and 100 Hz, and the shares of spectral power in the 0.5–5 Hz and 5–50 Hz bands.

$$I_{\text{rms}} = \sqrt{\frac{1}{L}\sum_{k=1}^{L} {x_k}^2} \qquad (9)$$

TABLE III. FEATURES EXTRACTED FROM EACH ONE-SECOND WINDOW

No.	Feature	What it indicates
1.	Mean	average current (load level)
2.	RMS	effective current, includes the ripple
3.	Std. deviation	size of the fluctuations
4.	Peak	largest sample in the window
5.	Peak-to-peak	maximum minus minimum
6.	Crest factor	peak / RMS, high for spikes
7.	Kurtosis	spikiness of the distribution
8.	Dominant frequency	frequency of the largest FFT peak (0.75–100 Hz)
9.	Dominant amplitude	height of that peak
10.	Energy 0.5–5 Hz	share of spectral power in the band
11.	Energy 5–50 Hz	share of spectral power in the band

3. Random Forest Classification

A Random Forest of 100 decision trees assigns each window to one of three classes. The training data are 1200 simulated windows, 400 per class, with the severity s drawn uniformly from 0.25 to 1. Each window comes from a motor with slightly different parameters (no-load current ±15%, speed ±10%, noise 0.6–1.5 times nominal) so that the model does not memorize one simulated unit. Of these windows, 900 (75%) train the forest and 300 (25%) are held out for testing. The class probability is the average of the trees' class probabilities, which for fully grown trees is the fraction of trees that vote for the class (Equation (10), where n_c is the number of the T = 100 trees that vote for class c). To avoid flicker, the probabilities are smoothed with an exponential moving average (Equation (11)) with α = 0.25, where p_k is the forest output and q_k the smoothed probability of update k, and the displayed label changes only when the smoothed probability of the new class reaches 0.6.

$$P(c) = \frac{n_c}{T} \qquad (10)$$

$$q_k = (1-\alpha)\, q_{k-1} + \alpha\, p_k \qquad (11)$$

4. Real-Time Web Dashboard

A FastAPI server streams 20 frames per second over a WebSocket. Each frame carries 50 new samples of every stage of the chain, the eleven features, the 40 Hz spectrum, the class probabilities and the votes of the 100 trees. A Next.js dashboard draws the current waveform, the spectrum, a 60 s prediction timeline and the verdict with its confidence, and further pages show a 3D model of the bench, the live circuit schematic and the processing pipeline. The fault buttons change only the hidden load and rub parameters of the simulator; the classifier never receives the selected condition, so the displayed prediction is always inferred from the sampled current. The same server is configured to run on a serverless platform, where each browser session drives its own simulated motor.

IV. RESULTS

The system was evaluated in the software-in-the-loop simulation described above. All numbers in this section are measured from that simulation with the scripts that accompany the project, and they describe a model of the circuit and the motor, not a physical motor.

A. Behavior of the Analog Chain

Fig. 5 follows a friction fault through the chain. The current spikes of the rub appear at the shaft frequency of about 3 Hz and are carried through the amplifier and the filter to the ADC without distortion. The free-running motor draws 0.33 A (136 counts, 196 RPM), the motor at 0.7 overload severity draws 0.80 A (328 counts, 130 RPM) and the motor in the friction fault at severity 0.6 draws 0.49 A (201 counts, 178 RPM).

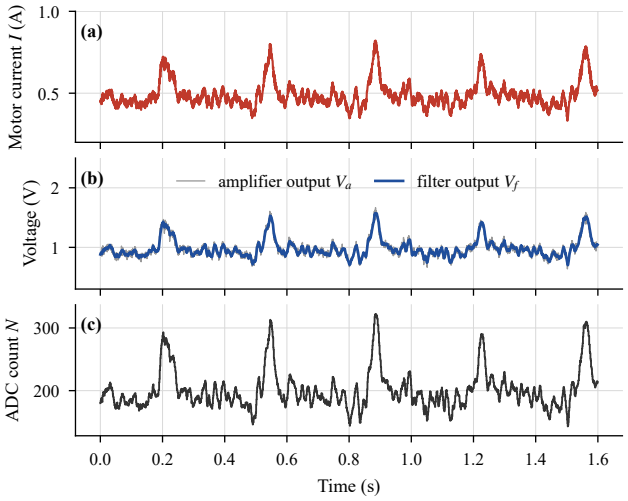


Fig. 5. Signals at each stage of the chain for a friction fault: (a) motor current, (b) amplifier and filter outputs, (c) ADC counts

To measure what the filter contributes, the Sallen-Key stage was bypassed in the simulation. The error between the current reconstructed from the ADC and the true low-frequency motor current rose from 3.2 mA rms with the filter to 16.1 mA without it, a factor of five, and the residual content above 60 Hz rose from 2.9 mA to 15.6 mA. When the classifier trained with the filter was fed the unfiltered data, its accuracy fell from 100% to 92.3%.

B. Fault Signatures

Fig. 6 shows three seconds of current for each condition and Fig. 7 the spectra of one-second windows. Overload raises the mean current from 0.33 A to 0.80 A and slows the shaft, which lowers the pulley frequency from 3.3 Hz to 2.2 Hz. Friction raises the mean only to 0.49 A but adds sharp once-per-revolution spikes. In the spectrum they appear as a series of peaks at the shaft frequency and its harmonics, with the fundamental near 3 Hz reaching 68 mA against 14 mA for the free-running motor. The crest factor rises from 1.07 to 1.55 and the kurtosis from 1.8 to 4.9, while 59% of the spectral power lies in the 5–50 Hz band against 1.6% when free running.

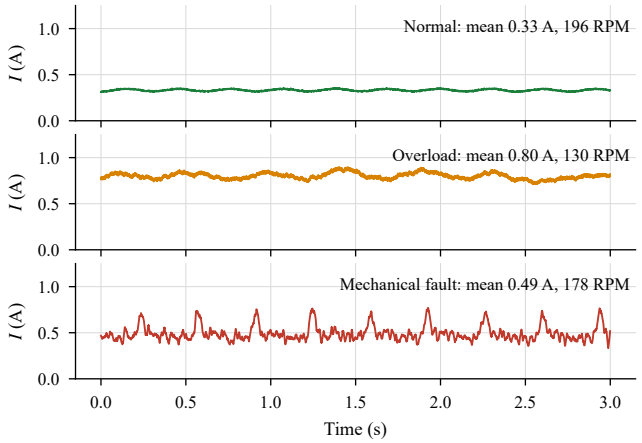


Fig. 6. Motor current reconstructed from the ADC counts for the three conditions

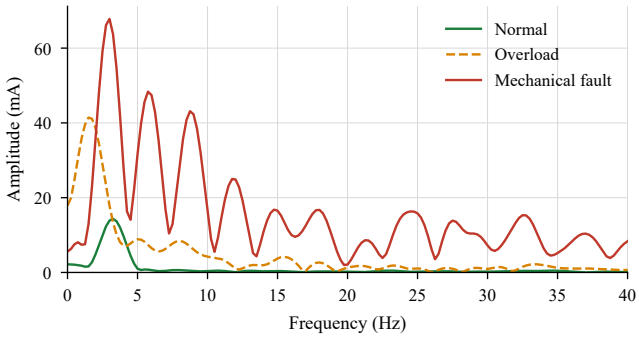


Fig. 7. Amplitude spectra of one-second windows for the three conditions

Fig. 8 shows which features the Random Forest uses. It relies mainly on the energy in the 5–50 Hz band, the crest factor and the peak-to-peak value, the features that react to spikes, together with the mean current and the standard deviation, which react to load. The dominant frequency and amplitude contribute least.

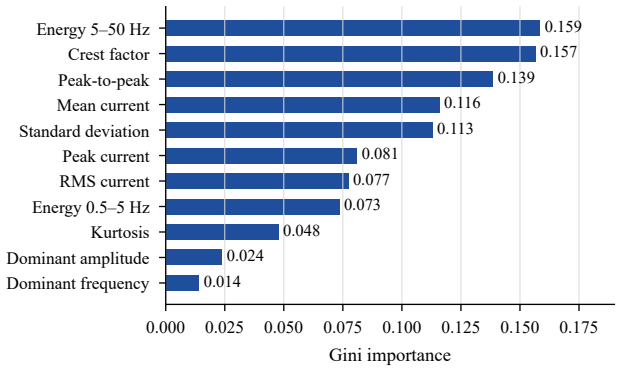


Fig. 8. Feature importance of the Random Forest

C. Classification Performance

On the 300 held-out windows the Random Forest classified every window correctly: 100 of 100 for each of the three classes, with no confusion between any pair. To check that the result does not depend on one split, 450 further windows were generated with new random seeds, and the accuracy was again 100%. Faults were also found at severities below the training range of 0.25: for severities between 0 and 0.1, overload was recognized in 96.7% and friction in 98.3% of 60 windows each. A score of 100% is to be expected when one generator produces both the training and the test data and the classes differ strongly, so the next experiments stress the model.

D. Model Comparison and Robustness

Table IV compares the Random Forest with four other classifiers trained on the same 900 windows. All five reach 99.7–100% on the held-out windows, but under stress they separate. With the amplifier noise multiplied by eight, the Random Forest keeps 99.3% while the SVM drops to 71.3% and k-NN to 75.0%. When the motor itself differs from the training units (no-load current ±40%, load coefficient ±25%), every model stays above 97%. With this unit shift and four times the noise, logistic regression is best with 100% and the Random Forest reaches 91.0%. Tested alone at higher noise, the Random Forest stayed at 100% up to four times the nominal noise and fell to 70.4% at sixteen times.

TABLE IV. ACCURACY (%) OF FIVE CLASSIFIERS UNDER NOMINAL AND STRESSED CONDITIONS

Classifier	Held-out	Noise ×8	Unit shift	Shift + noise ×4
Random Forest	100.0	99.3	99.3	91.0
Decision tree	99.7	95.7	97.3	94.0
SVM (RBF)	100.0	71.3	99.7	94.3
k-NN (k = 5)	100.0	75.0	99.7	98.0
Logistic regression	100.0	93.3	100.0	100.0

The task is therefore almost linearly separable in these features, and a linear model is a strong, cheap baseline. The Random Forest is kept because it tolerates noise and needs no feature scaling and because its importance scores explain the decision, but the best classifier for a real motor has to be decided with real data.

E. Real-Time Behavior

Fig. 9 shows the classifier following a scripted sequence of conditions. After a fault is injected the smoothed probability crosses 0.6 and the label changes after 0.87 ± 0.06 s on average for friction (severity 0.6) and 1.29 ± 0.10 s for overload (severity 0.7), over 30 trials each (mean ± standard deviation). At severity 0.3 the means are 0.93 s and 1.17 s, and every one of the 120 injected faults was detected. The delay is dominated by the one-second window and the 0.4 s mechanical response, not by computation: extracting the features takes 0.22 ms and the forest 3.4 ms (median on the development laptop), about 7% of the 50 ms update interval. In 12 min of simulated healthy operation (14,400 updates of 12 different motors) the label never left Normal, so there was no false alarm.

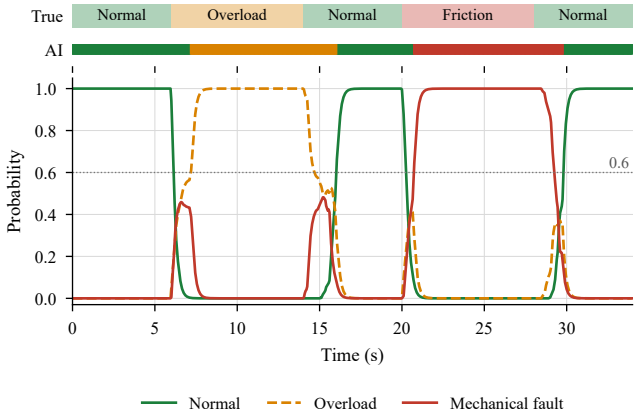


Fig. 9. Classifier response to a scripted sequence of injected faults

F. Limitations

The results are for a simulated motor and circuit. A real motor adds supply ripple, brush wear, temperature drift and its own signature for a given obstruction, so the accuracy on hardware will be lower. Features based on absolute current depend on the individual motor and should be recalibrated for each unit. The component values, the sampling chain and the software, however, are exactly those that will be built.

V. CONCLUSION

This study presents the design and simulation of an AI-based motor fault detector that senses only the motor current. A 0.1 Ω shunt, an LM358 differential amplifier (gain 20) and a unity-gain Sallen-Key filter (232 Hz) condition the signal for the 10-bit ADC of an Arduino Uno, and a Random Forest classifies eleven features of each one-second window as normal, overloaded or mechanically faulty. In the software-in-the-loop evaluation the classifier was correct on all 300 held-out windows, detected faults after 0.87–1.29 s on average with no false alarm in 12 min of healthy operation, and needed about 3.6 ms per update. Removing the analog filter increased the current-measurement error five-fold and reduced accuracy to 92.3%, which confirms that analog conditioning, and not only the classifier, determines the quality of the diagnosis. The comparison with other classifiers showed that a simple linear model is a strong baseline under combined stress, so the final choice should be made on real data.

Future work will build the circuit on a breadboard, record real windows for each condition, and retrain and re-evaluate the model. It will also add a Hall-effect current sensor for galvanic isolation, more fault classes such as bearing wear and supply faults, and inference on the microcontroller itself. The idea also scales to industrial machines: a companion simulator, Vigil [18], applies embedded real-time firmware to an induction motor with four sensors, and combining its scheduling and fault-detection logic with the analog front-end described here is a natural next step.

REFERENCES

[1] A. K. S. Jardine, D. Lin, and D. Banjevic, “A review on machinery diagnostics and prognostics implementing condition-based maintenance,” Mechanical Systems and Signal Processing, vol. 20, no. 7, pp. 1483–1510, Oct. 2006.

[2] S. Nandi, H. A. Toliyat, and X. Li, “Condition monitoring and fault diagnosis of electrical motors—A review,” IEEE Transactions on Energy Conversion, vol. 20, no. 4, pp. 719–729, Dec. 2005.

[3] M. Riera-Guasp, J. A. Antonino-Daviu, and G.-A. Capolino, “Advances in electrical machine, power electronic, and drive condition monitoring and fault detection: State of the art,” IEEE Transactions on Industrial Electronics, vol. 62, no. 3, pp. 1746–1759, Mar. 2015.

[4] M. E. H. Benbouzid, “A review of induction motors signature analysis as a medium for faults detection,” IEEE Transactions on Industrial Electronics, vol. 47, no. 5, pp. 984–993, Oct. 2000.

[5] W. T. Thomson and M. Fenger, “Current signature analysis to detect induction motor faults,” IEEE Industry Applications Magazine, vol. 7, no. 4, pp. 26–34, Jul. 2001.

[6] Y. Lei et al., “Applications of machine learning to machine fault diagnosis: A review and roadmap,” Mechanical Systems and Signal Processing, vol. 138, p. 106587, Apr. 2020.

[7] S. Zhang, S. Zhang, B. Wang, and T. G. Habetler, “Deep learning algorithms for bearing fault diagnostics—A comprehensive review,” IEEE Access, vol. 8, pp. 29857–29881, 2020.

[8] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[9] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, Oct. 2011.

[10] Texas Instruments, “LM158, LM258, LM358, LM2904 dual operational amplifiers,” Datasheet SLOS068, Dallas, TX, USA.

[11] P. Horowitz and W. Hill, The Art of Electronics, 3rd ed. Cambridge, U.K.: Cambridge University Press, 2015.

[12] R. P. Sallen and E. L. Key, “A practical method of designing RC active filters,” IRE Transactions on Circuit Theory, vol. 2, no. 1, pp. 74–85, Mar. 1955.

[13] J. Karki, “Active low-pass filter design,” Texas Instruments, Dallas, TX, USA, Application Report SLOA049B, Sep. 2002.

[14] C. E. Shannon, “Communication in the presence of noise,” Proceedings of the IRE, vol. 37, no. 1, pp. 10–21, Jan. 1949.

[15] F. J. Harris, “On the use of windows for harmonic analysis with the discrete Fourier transform,” Proceedings of the IEEE, vol. 66, no. 1, pp. 51–83, Jan. 1978.

[16] Microchip Technology, “ATmega48A/PA/88A/PA/168A/PA/328/P data sheet,” Document DS40002061, Chandler, AZ, USA.

[17] I. Fette and A. Melnikov, “The WebSocket protocol,” Internet Engineering Task Force, RFC 6455, Dec. 2011.

[18] Sidhant, “Vigil: Design and simulation of a real-time embedded predictive maintenance system for industrial motor fault detection,” unpublished project software, 2026.